Autonomous Drone Inspection of Offshore Wind Farms

Alessandro Massari, Rong Zeng -
Planning & Reasoning A.Y. 2025/26

Table of contents

Autonomous Path Planning and Reasoning for Drone Offshore Wind Farm Inspection

A Hybrid Approach Integrating Reinforcement Learning with PDDL and IndiGolog

Project Goal

- **Automation:** Minimize human intervention in high-risk offshore environments through autonomous drone fleet management.
- **Optimization:** Solve complex path-planning tasks by balancing inspection coverage with strict energy (battery) constraints.
- **Hybrid Framework:** Integrating a Reinforcement Learning (RL) component for low-level decision-making with a formal Planning and Reasoning framework for high-level logic.

PDDL domain 1

TYPES

drone location - object

base – location

PREDICATES

(at ?d - drone ?l - location)

(is_inspected ?l - location)

(is_turbine ?t - location)

(severe_fault ?t - location)

(severe_detected ?t - location)

(fault_handled ?t - location)

FUNCTIONS

(fly-cost ?from ?to - location)
Each pair of locations has an an associated travel cost representing the energy required to fly between them.

(inspection-cost)
Represents the fixed amount of battery consumed by perfoming a turbine inspection action.

(battery ?d - drone)
Represents the current battery level of the drone. Decrease accordingly after each flight or inspection action. It's checked before each action.

PDDL domain 2

ACTIONS	DESCRIPTION	PRECONDITIONS	EFFECTS
FLY	Allow the drone to move between locations	at departure location, enough battery, no severe fault to handle	at new location, battery level decrease accordingly to fly-cost
INSPECT	Inspects wind turbine	at turbine and have enough battery to perform the inspection	Inspected(turbine), emergency procedure if severe fault, battery decreases
BACKTOBASE	Emergency safety procedure triggered by severe fault discovery or low battery level	at turbine, turbine inspected, severe fault discovered and enough battery to come back	Drone moves back to the base, battery decreases, severe fault marked as handled
RECHARGE	The drone recharges its battery at the base station.	at base and battery not full (90%)	Battery level is restored to the maximum

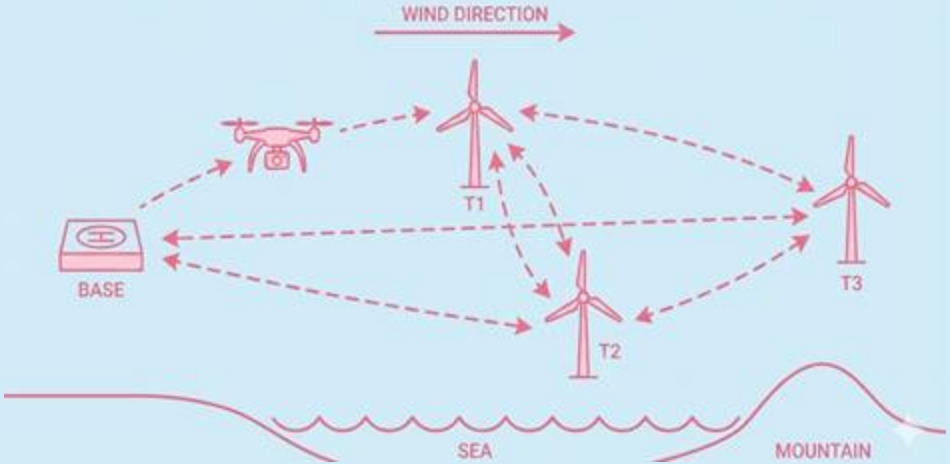
PDDL Problems: P1

Goal: A single drone must check all wind turbines in the farm

We have :

1 drone 

3 turbine 



search algorithm	planning time [ms]	solution length	costs [battery]
GBFS + H_add	242	9	95
A* + H_add	**204**	**7**	**75**

PDDL Problems: P1

```
Found Plan:
0.0: (fly drone1 Base t3)
1.0: (inspect drone1 t3)
2.0: (fly drone1 t3 t2)
3.0: (inspect drone1 t2)
4.0: (fly drone1 t2 Base)
5.0: (recharge drone1 Base
6.0: (fly drone1 Base t1)
7.0: (inspect drone1 t1)
8.0: (fly drone1 t1 Base)

Plan-Length:9
```

GBFS + H_add plan

```
Found Plan:
0.0: (fly drone1 Base t1)
1.0: (inspect drone1 t1)
2.0: (fly drone1 t1 t3)
3.0: (inspect drone1 t3)
4.0: (fly drone1 t3 t2)
5.0: (inspect drone1 t2)
6.0: (fly drone1 t2 Base)

Plan-Length:7
```

A* + H_max plan

*ENHSP-20 was used as the solver for all problem instances.

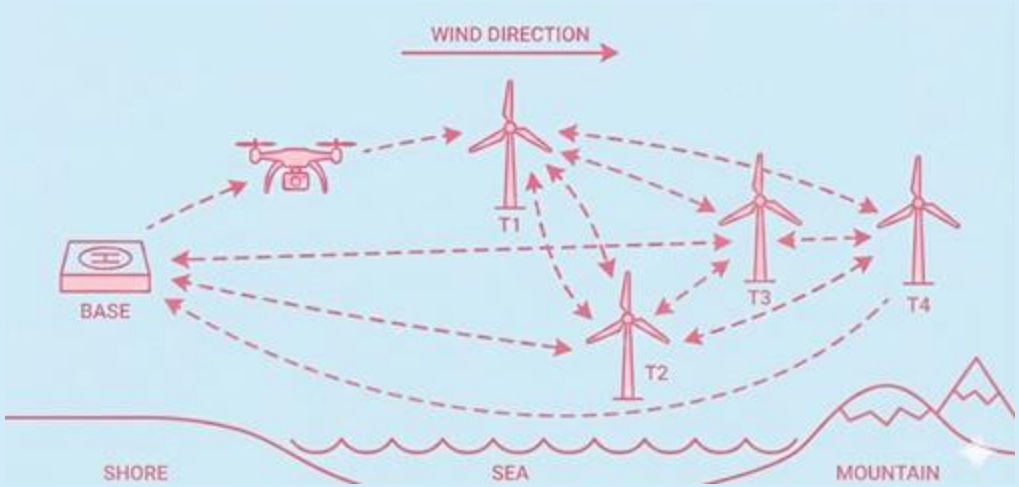
PDDL Problems: P2

Goal: Two drones must check all wind turbines in the farm

We have :

2 drone

4 turbine



search algorithm	planning time [ms]	solution length	costs [battery]
GBFS + H_add	**308 ms**	16	200
A* + H_max	1465 ms	**13**	**150**

PDDL Problems: P2

```
Found Plan:
0.0: (fly drone2 Base t3)
1.0: (inspect drone2 t3)
2.0: (fly drone2 t3 t1)
3.0: (inspect drone2 t1)
4.0: (fly drone1 Base t3)
5.0: (fly drone2 t1 Base)
6.0: (recharge drone2 Base)
7.0: (fly drone1 t3 t2)
8.0: (inspect drone1 t2)
9.0: (fly drone1 t2 Base)
10.0: (fly drone2 Base t3)
11.0: (fly drone2 t3 t4)
12.0: (inspect drone2 t4)
13.0: (fly drone2 t4 t3)
14.0: (fly drone2 t3 Base)
15.0: (recharge drone2 Base)
```

GBFS + H_add plan

```
Found Plan:
0.0: (recharge drone1 Base)
1.0: (fly drone1 Base t3)
2.0: (fly drone2 Base t1)
3.0: (inspect drone2 t1)
4.0: (fly drone1 t3 t4)
5.0: (fly drone2 t1 t2)
6.0: (inspect drone1 t4)
7.0: (fly drone1 t4 t3)
8.0: (inspect drone2 t2)
9.0: (fly drone2 t2 Base)
10.0: (inspect drone1 t3)
11.0: (fly drone1 t3 Base)
12.0: (recharge drone1 Base)
```

A* + H_max plan

PDDL Problems: P3

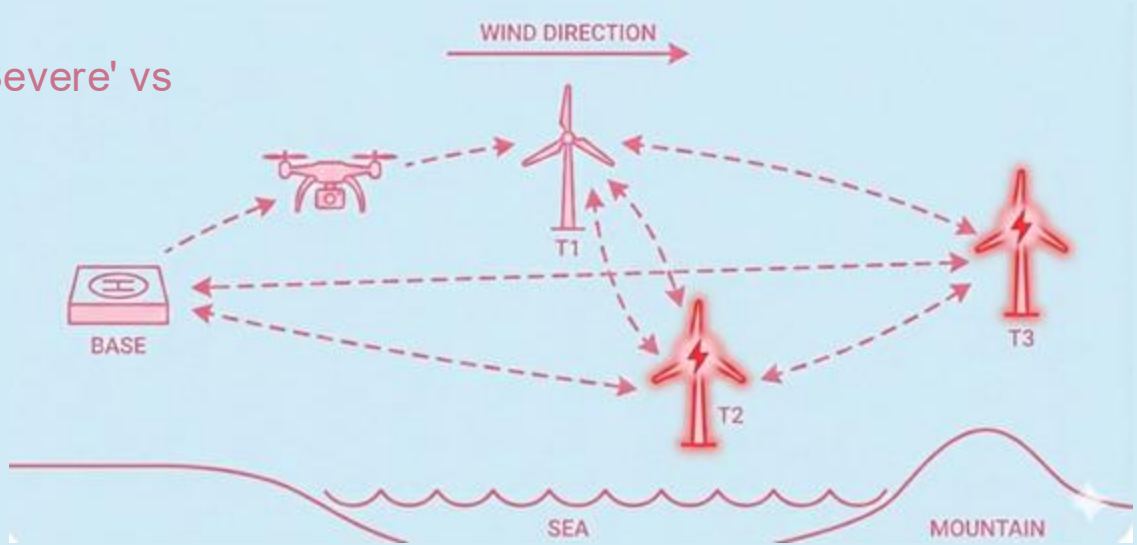
Goal: Handle the discovery of 'Severe' vs 'Minor' issues during inspect

We have :

3 drone

3 turbine

2 ‘severe fault’



search algorithm	planning time [ms]	solution length	costs [battery]
GBFS + H_add	**286 ms**	10	**105**
A* + H_max	3482 ms	10	125

PDDL Problems: P3

```
Found Plan:
0.0: (fly drone2 Base t3)
1.0: (fly drone3 Base t1)
2.0: (inspect drone2 t3)
3.0: (backtobase drone2 t3 Base)
4.0: (inspect drone3 t1)
5.0: (backtobase drone3 t1 Base)
6.0: (fly drone1 Base t3)
7.0: (fly drone1 t3 t2)
8.0: (inspect drone1 t2)
9.0: (fly drone1 t2 Base)
```

GBFS + H_add plan

```
Found Plan:
0.0: (fly drone1 Base t1)
1.0: (fly drone3 Base t3)
2.0: (fly drone2 Base t1)
3.0: (inspect drone3 t3)
4.0: (fly drone1 t1 t2)
5.0: (inspect drone2 t1)
6.0: (inspect drone1 t2)
7.0: (backtobase drone3 t3 Base)
8.0: (backtobase drone2 t1 Base)
9.0: (fly drone1 t2 Base)
```

A* + H_max plan

Indigolog

Indigolog: domain

```
% FLUENTS

prim_fluent(current_location).
prim_fluent(inspected(T)) :-
is_turbine(T).
prim_fluent(severe_fault).
prim_fluent(battery_level).


% PRIMITIVE ACTIONS

prim_action(fly(L)) :- location(L).
prim_action(inspect(T)) :-
is_turbine(T).
prim_action(charge(_Drone)).
prim_action(report(_Drone)).
```

```
% SSA for severe_fault logic to detect faults
causes_val(inspect(t2), severe_fault,
true, true).

causes_val(report(_), severe_fault,
false, true).

% SSA for battery_level
causes_val(fly(To), battery_level,
NewVal, and(route(current_location, To,
Cost), NewVal is battery_level -
Cost)).

causes_val(inspect(_), battery_level,
NewVal, NewVal is battery_level - 5).

causes_val(charge(_), battery_level,
100, true).
```

Indigolog: The tasklist

LEGALITY TASK

Starting with battery level at 50%, check if the drone is able to check all or a subset of the wind-farm turbines avoiding discharge below the battery threshold level.

INTERACTIVE LEGALITY TASK

Check the actions inserted by the user

PROJECTION TASK:

```
[fly(t2), inspect(t2),
fly(t1), inspect(t1),
fly(base), fly(t3),
inspect(t3),fly(base)]

?(and(current_location
= base, batter_level
>= 30))
```

Indigolog: tasks' results

LEGALITY TASK:

set battery level to 50%, check if the drone is able to check all or a subset of the wind-farm turbines avoiding discharge below the battery threshold level.

FALSE!

PROJECTION TASK:

[fly(t2) , inspect(t2), fly(t1), inspect(t1), fly(base), fly(t3), inspect(t3), fly(base)]

?(and(current_location = base, batter_level >= 30))

FALSE!

100 - (40+5+10+5+20) = 20 < 30!

Indigolog: Controllers

BASIC CONTROLLER:
Pick a turbine and inspect

No battery management!
No severe fault handling!

```
 ----  OFFSHORE WIND FARM  ----
Controllers available: [basic,smart]
Select controller: basic
|: .
Executing controller: *basic*

fly(t1)
inspect(t1)
fly(t2)
inspect(t2)
fly(t3)
inspect(t3)
false.
```

```
 ----  OFFSHORE WIND FARM  ----
Controllers available: [basic,smart]
Select controller: smart.
Executing controller: *smart*

start_interrupts
fly(t1)
inspect(t1)
fly(t2)
inspect(t2)
>>SEVERE FAULT DETECTED! Report to Base...
fly(base)
>>LOW BATTERY!
>>Already at base...
charge(drone)
>>Battery charged.
report(drone)
fly(t3)
inspect(t3)
stop_interrupts

11 actions.
true .
```

SMART CONTROLLER:
Introduction of prioritized interrupts

Battery correctly managed!
Severe fault handled as in PDDL!

Conclusions

PDDL

Formal definition of state space and problems. Complexity idea avoiding to run expensive HW

INDIGOLOG

Reasoning for achieve complex tasks in different scenarios

WHAT'S NEXT

Compare logical control performance against RL ones and explore hybrid architectures

Thanks.

Contacts: Alessandro Massari, 荣曾 (Rong Zeng)
Resources: GitHub repository